Where Losses Come From

Deep Learning course notes Sources: listed under References

What this file is really about:

$$\text{a model says which values are likely} \rightarrow \text{likelihood} \rightarrow \text{negative log} \rightarrow \text{a loss} \rightarrow \text{priors and robust}$$

How to use this file. Part A derives the common losses. Part B shows the code. Part C is practice with answers.

Step-up path

You need. Probability densities and logarithms.

1. See a model as a distribution over outputs.
2. Write the likelihood and take its negative log.
3. Recover squared error and cross-entropy.
4. Let the model learn its noise.
5. Add priors and robust noise.

You can stop when you can derive a loss from an assumption about noise.

Contents

Part A Maths From the Ground Up

A1 A loss is a modelling choice

A network does not have to output one number. It can output the parameters of a distribution: a mean, a variance, or class probabilities. Training then asks that the observed data be likely under that distribution.

A2 Likelihood of independent data

If examples are independent, their joint probability is a product:

$$p(\mathcal{D} \mid \theta) = \prod_i p(y_i \mid x_i, \theta).$$

Products underflow and are awkward to differentiate, so we take logs. Minimising the **negative log-likelihood** turns the product into a sum, one term per example.

A3 Squared error is a Gaussian assumption

Suppose $y \sim \mathcal{N}(\mu(x), \sigma^2)$. Then

$$-\log p(y \mid x) = \frac{(y - \mu(x))^2}{2\sigma^2} + \tfrac{1}{2}\log(2\pi\sigma^2).$$

With fixed σ, the second term is constant, so minimising it is the same as minimising squared error.

A4 Cross-entropy is a categorical assumption

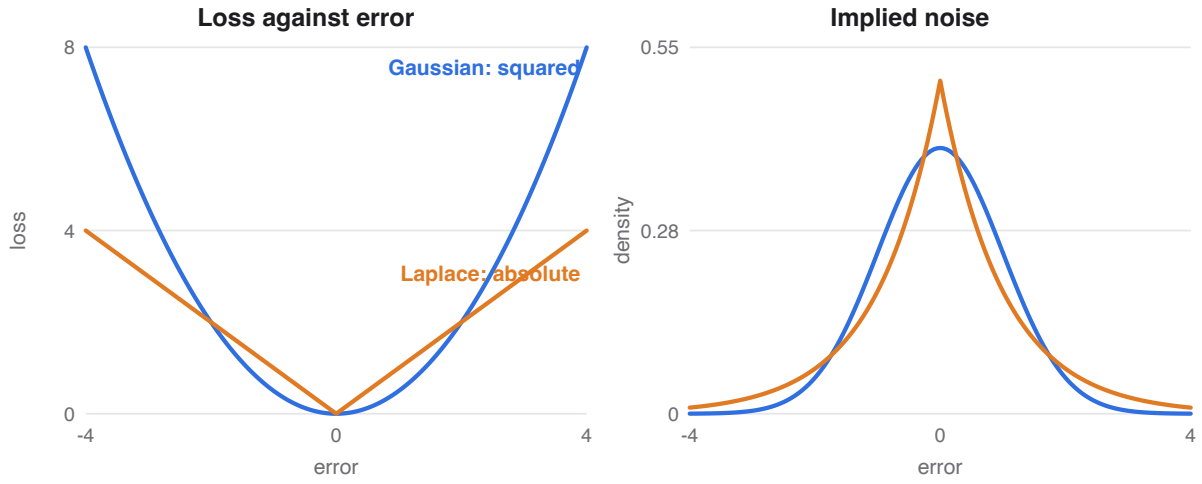
Softmax turns scores into probabilities, and the negative log-likelihood of the true class is the cross-entropy:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad L = -\log p_y.$$

Its gradient with respect to the scores is $p - e_y$, predicted minus true.

A5 Learning the noise

If the network also outputs $\sigma(x)$, it can say where it is unsure. For a given error e, the loss $\frac{e^2}{2\sigma^2} + \log \sigma$ is smallest at $\sigma = |e|$. The model pays for confidence that turns out to be wrong.



A Gaussian model punishes large errors quadratically. A Laplace model, whose negative log gives absolute error, punishes them linearly and so resists outliers.

A6 Priors and MAP

A prior adds a term to the loss. With a Gaussian prior on the weights,

$$\text{loss} = \text{NLL}(\theta) + \frac{\lambda}{2}\|\theta\|^2.$$

For a single mean with prior $\mathcal{N}(0, \tau^2)$ and known noise σ^2, MAP shrinks the sample mean towards zero:

$$\hat{\mu} = \frac{n\bar{x}/\sigma^2}{n/\sigma^2 + 1/\tau^2}.$$

More data reduce the influence of the prior.

A7 Robust observation models

A Laplace likelihood gives absolute error, and a Student-t likelihood gives heavy tails. Both reduce the influence of outliers, and are simply different choices of what noise looks like.

A8 Binary cross-entropy, derived

A Bernoulli outcome has $p(y \mid \pi) = \pi^y (1-\pi)^{1-y}$. Taking the negative log for one example gives

$$-\log p = -y \log \pi - (1-y)\log(1-\pi).$$

Substitute $\pi = \sigma(z)$. Because $\log \sigma(z) = -\log(1+e^{-z})$ and $\log(1-\sigma(z)) = -z - \log(1+e^{-z})$, the loss in terms of the logit is

$$L(z, y) = (1-y)z + \log(1+e^{-z}).$$

Its derivative is remarkably simple:

$$\frac{\partial L}{\partial z} = \sigma(z) - y.$$

A9 Why squared error is poor with a sigmoid

Compare gradients for a confidently wrong prediction, with $y = 1$ and z very negative.

- Cross-entropy: $\sigma(z) - 1 \approx -1$. The gradient stays large, so learning continues.
- Squared error: $2(\sigma(z) - 1)\sigma'(z) \approx -2\sigma'(z) \approx 0$. The gradient vanishes exactly when the model is most wrong.

A10 Absolute error is a Laplace assumption

With $p(y \mid \mu) = \frac{1}{2b}\exp(-|y-\mu|/b)$, the negative log-likelihood is $|y-\mu|/b + \log 2b$. So minimising absolute error is maximum likelihood under Laplace noise, whose heavier tails make it robust to outliers. The best constant prediction is the median, because the gradient $\sum_i \operatorname{sign}(\mu - y_i)$ is zero when equal numbers of points lie on each side.

A11 A general recipe

To design a loss for any target:

1. Pick a distribution that can produce the target: Gaussian for real values, Bernoulli for binary, categorical for classes, Poisson for counts.
2. Make the network output the distribution's parameters, transformed to be valid.
3. Minimise the negative log-likelihood.

For counts with a rate $\lambda = e^z$, the Poisson negative log-likelihood is $\lambda - yz + \log y!$.

Part B In Code

B1 Gaussian negative log-likelihood

```
nll = 0.5 * ((y - mu) ** 2 / var + torch.log(2 * math.pi * var)).mean()
```

B2 Work with logits

```
loss = F.cross_entropy(logits, y)   # applies log-softmax inside, stably
```

B3 Comparing losses and learning the noise

A complete experiment: the noise on this data grows with x, and the network learns that.

```
import torch
from torch import nn

torch.manual_seed(0)
x = torch.linspace(-3, 3, 800).unsqueeze(1)
y = torch.sin(x) + (0.05 + 0.25 * x.abs()) * torch.randn_like(x)

net = nn.Sequential(nn.Linear(1, 64), nn.Tanh(), nn.Linear(64, 64), nn.Tanh(), nn.Linear(64
opt = torch.optim.Adam(net.parameters(), lr=3e-3)

for step in range(1500):
    mu, log_var = net(x).chunk(2, dim=1)
    loss = (0.5 * ((y - mu) ** 2 / log_var.exp() + log_var)).mean()
    opt.zero_grad(); loss.backward(); opt.step()

sd = (0.5 * net(x).chunk(2, dim=1)[1]).exp().detach().squeeze()
print('predicted noise at x=-3, 0, 3:', [round(sd[i].item(), 2) for i in (0, 400, 799)])
assert sd[0] > 2 * sd[400] and sd[799] > 2 * sd[400]
```

B4 Check the formulas against the library

```
import torch
from torch.nn import functional as F

z = torch.tensor([-2.0, 0.5, 3.0]); y = torch.tensor([1.0, 0.0, 1.0])
by_hand = ((1 - y) * z + torch.log1p(torch.exp(-z))).mean()
assert abs(by_hand - F.binary_cross_entropy_with_logits(z, y)) < 1e-6

z = torch.tensor([[1.0, -1.0, 0.5]]); t = torch.tensor([2])
assert abs(-F.log_softmax(z, 1)[0, 2] - F.cross_entropy(z, t)) < 1e-6
print('formulas match')
```

Part C Practice

P1. With $\sigma = 1$, $y = 3$ and $\mu = 1$, find the squared part of the Gaussian NLL.

$(3 - 1)^2/2 = 2.$

P2. Scores give the true class a probability of 0.2. Find the cross-entropy.

$-\ln 0.2 = 1.609.$

P3. An error of $e = 2$. What σ minimises the loss with learned noise?

$\sigma = 2.$

P4. Which noise model gives absolute error?

The Laplace distribution.

P5. Show that $\partial L/\partial z = \sigma(z) - y$ for binary cross-entropy at $y = 0$.

With $y = 0$, $L = z + \log(1 + e^{-z})$. Then $L' = 1 - e^{-z}/(1 + e^{-z}) = 1 - (1 - \sigma(z)) = \sigma(z)$.

P6. What is the best constant prediction under Pinball loss with $\tau = 0.9$?

The 90th percentile. Asymmetric absolute loss has the quantile as its minimiser.